Навигация и общие данные



Navigation 3, ViewModel экрана и репозиторий

Разработка мобильных приложений на Android

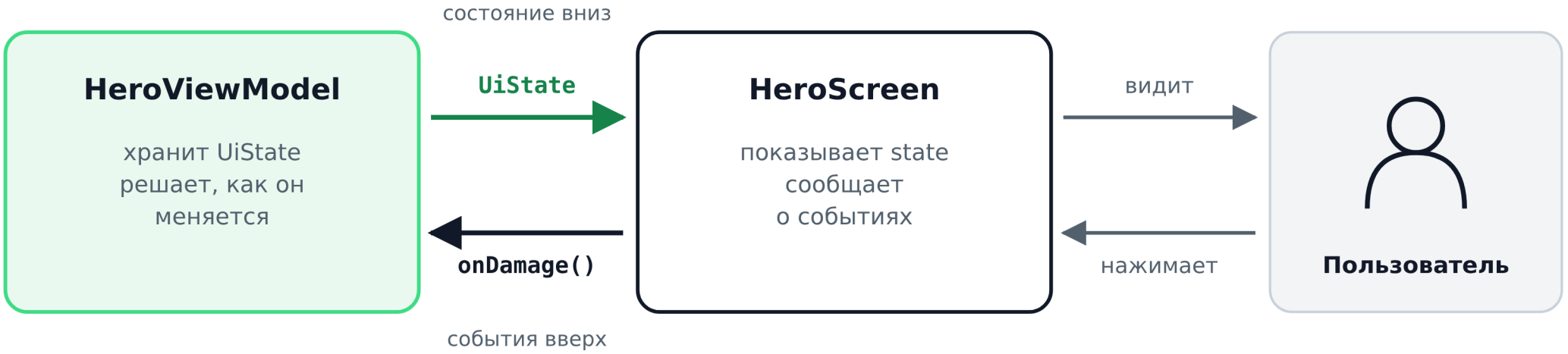
От одного экрана к приложению

Что должно стать понятным

- Как устроен стек экранов и переходы между ними
- Почему у каждого экрана своя ViewModel и как передать ей id
- Как два экрана работают с одними данными
- Где создать объект, который живёт всё время работы приложения

Результат: список предметов и карточка, изменения на карточке видны в списке.

Повторим: однонаправленный поток данных



1. нажал «Получить урон»
2. вызван onDamage()
3. новый UiState: hp = 90
4. экран показал 90

Сегодня по той же схеме устроим переходы и общие данные.

Что уже умеем

Часть	Отвечает за
HeroViewModel	Хранит UiState экрана и правила его изменения
HeroUiState	Неизменяемый снимок данных экрана
HeroRoute	Получает ViewModel и связывает её с экраном
HeroScreen	Показывает state, сообщает о событиях
Inventory, ItemRow	Список на LazyColumn, только параметры

Сегодня в таблице появятся навигация и репозиторий.

Нажали на зелье

Хотим карточку предмета:
описание, сила, «Использовать».

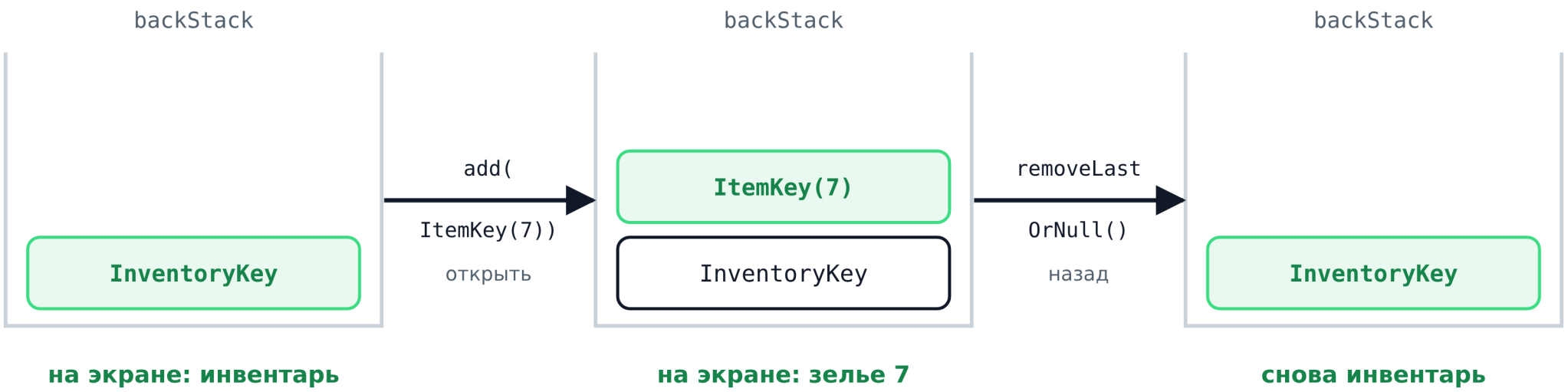
Показывать её поверх списка? Заменять список? Что будет по кнопке «назад»?

Какой экран открыт — тоже состояние

- Стек экранов — список, наверху текущий экран
- Открыть экран — добавить ключ в конец списка
- Назад — убрать последний ключ
- Библиотека навигации только показывает верх стека

Навигация — тот же UDF: состояние стека и события «открыть» и «назад».

Стек — это список ключей



Наверху стека — экран, который видит пользователь.

Navigation 3

- Библиотека навигации для Compose, стабильна с ноября 2025
- Стек хранит приложение, а не библиотека
- NavDisplay показывает верхний ключ стека
- entryProvider решает, какой экран показать для ключа

Ключ — какой экран. entryProvider — как его показать. NavDisplay — показать.

Зависимости

Артефакт	Зачем
androidx.navigation3:navigation3-runtime	Ключи, стек, entryProvider
androidx.navigation3:navigation3-ui	NavDisplay
androidx.lifecycle:lifecycle-viewmodel-navigation3	Своя ViewModel у каждого экрана
Плагин и библиотека kotlinx.serialization	Сохранение стека при повороте

Версии добавляем в libs.versions.toml, как остальные зависимости.

Ключи и стек

```
data object InventoryKey
data class ItemKey(val itemId: Int)

@Composable
fun HeroNavigation() {
    val backStack = remember {
        mutableStateListOf<Any>(InventoryKey)
    }
    // NavDisplay — на следующем слайде
}
```

Ключ — данные, а не экран.

В ItemKey только id, не сам предмет.

backStack — обычный список в Compose-состоянии.

Стартовый экран — первый элемент списка.

NavDisplay: стек → экран

```
NavDisplay(
    backStack = backStack,
    onBack = { backStack.removeLastOrNull() },
    entryProvider = entryProvider {
        entry<InventoryKey> {
            InventoryRoute(onOpenItem = { id ->
                backStack.add(ItemKey(id))
            })
        }
        entry<ItemKey> { key ->
            ItemRoute(itemId = key.itemId)
        }
    },
)
```

entry<T> связывает класс ключа с экраном.

key — сам ключ, из него берём id.

onBack вызывается по системной «назад».

Переходы — изменения списка. onBack передаём явно, чтобы это было видно.

NavHost и composable — это Navigation 2

Navigation 2 (Navigation Compose)	Navigation 3
NavHost	NavDisplay
composable<Inventory> { … }	entry<InventoryKey> { … } в entryProvider
navController.navigate(Item(7))	backStack.add(ItemKey(7))
navController.popBackStack()	backStack.removeLastOrNull()
backStackEntry.toRoute<Item>()	key в entry<ItemKey> { key -> }
Стек хранит NavController	Стек — наш список ключей
androidx.navigation:navigation-compose	androidx.navigation3:navigation3-ui

Нашли NavHost — это Navigation 2. В Navigation 3 стек — наш список.

Как работает «назад»

Что произошло	Что со стеком
Нажали на предмет	backStack.add(ItemKey(id))
Системная «назад» или жест	NavDisplay вызывает onBack → removeLastOrNull()
Кнопка «Назад» на экране	Тот же removeLastOrNull() через колбэк
«Назад» на первом экране	NavDisplay не перехватывает, приложение закрывается

Все переходы — изменения одного списка в одном месте.

Экран не знает о стеке

```
@Composable
fun InventoryRoute(
    onOpenItem: (Int) -> Unit,
) {
    val viewModel: InventoryViewModel = viewModel()
    InventoryScreen(
        state = viewModel.uiState,
        onItemClick = onOpenItem,
        onDamage = viewModel::onDamage,
    )
}
```

Переход — событие, как onDamage.

Route не знает, что откроется и как.

Куда переходить, решает только навигация. Экраны сообщают, что хотят.

Открыли зелье и повернули экран

**Карточка исчезла,
снова список.**

Где мы уже видели такое?

rememberNavBackStack: стек переживает поворот

```
@Serializable
data object InventoryKey : NavKey

@Serializable
data class ItemKey(val itemId: Int) : NavKey

@Composable
fun HeroNavigation() {
    val backStack = rememberNavBackStack(InventoryKey)
    NavDisplay(backStack = backStack, ...)
}
```

NavKey — метка: ключ можно сохранить.

@Serializable — плагин kotlinx. serialization.

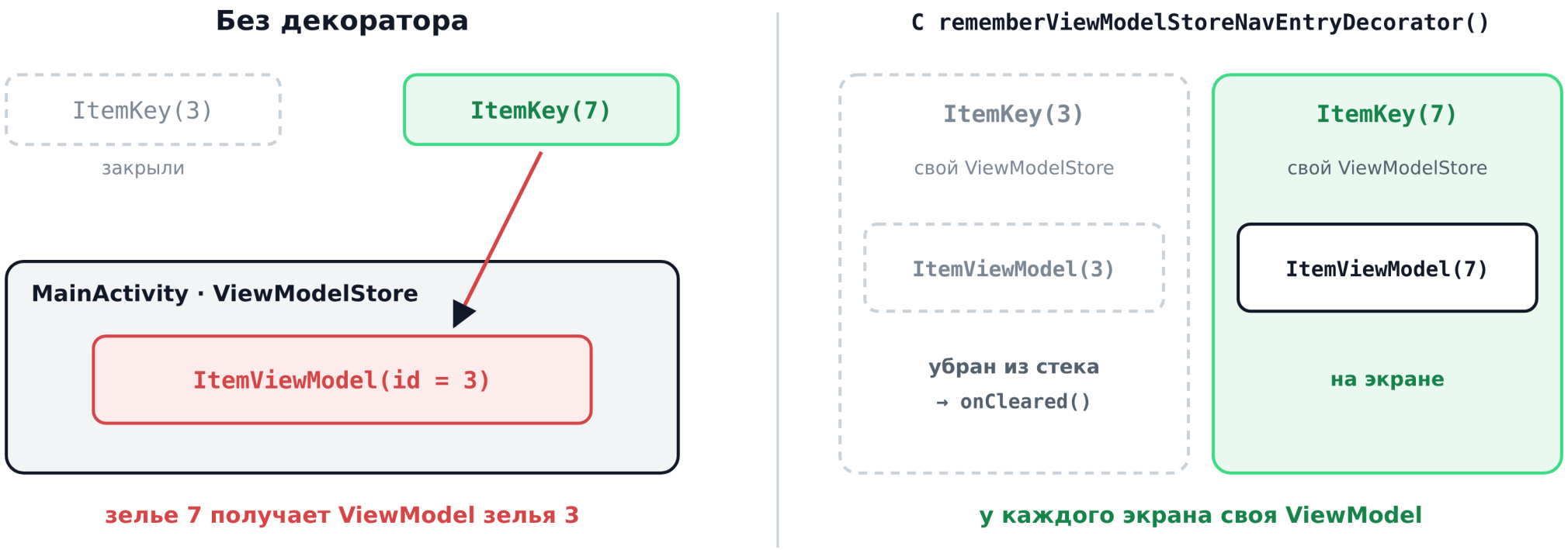
Стек переживает поворот и завершение процесса, как rememberSaveable.

Чья это ViewModel?

**Открыли зелье 3, вернулись,
открыли зелье 7.**

viewModel() в карточке вернёт новый объект или тот, что был у зелья 3?

Своя ViewModel у каждого экрана



Без декоратора все экраны делят ViewModelStore Activity.

Декораторы NavDisplay

```
NavDisplay(
    backStack = backStack,
    onBack = { backStack.removeLastOrNull() },
    entryDecorators = listOf(
        rememberSaveableStateHolderNavEntryDecorator(),
        rememberViewModelStoreNavEntryDecorator(),
    ),
    entryProvider = entryProvider { ... },
)
```

Первый — сохраняет rememberSaveable каждого экрана.

Второй — даёт экрану свой ViewModelStore.

Указали свои декораторы — первый нужно добавить явно, иначе он пропадёт.

Сколько живёт ViewModel экрана

Событие	ItemViewModel зелья 7
Открыли карточку	Создана
Повернули экран	Тот же объект
Поверх открыли другой экран	Жива: ключ ещё в стеке
«Назад» с карточки	onCleared(), объект удалён
Снова открыли зелье 7	Новый объект

ViewModel экрана живёт, пока его ключ в стеке.

Как передать ViewModel id

```
class ItemViewModel(
    private val itemId: Int,
) : ViewModel() {
    // ...
}

@Composable
fun ItemRoute(itemId: Int) {
    val viewModel = viewModel { ItemViewModel(itemId) }
    ItemScreen(state = viewModel.uiState, ...)
}
```

viewModel { } — фабрика: как создать объект, если его ещё нет.

Лямбда вызовется один раз для экрана.

Аргументы экрана приходят из ключа и передаются в конструктор ViewModel.

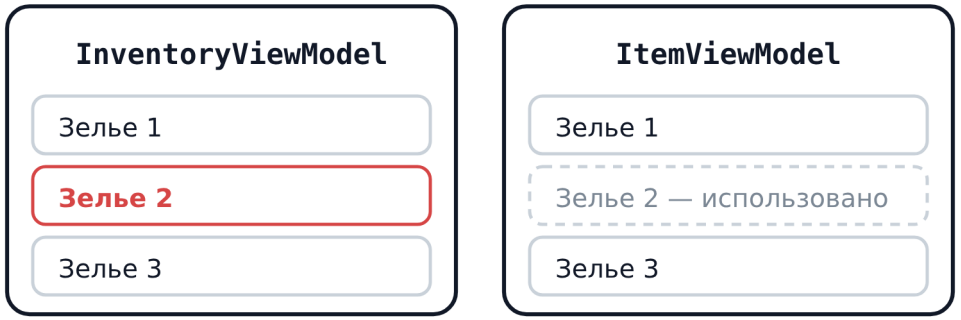
Использовали зелье на карточке

**Вернулись в список —
зелье на месте.**

Каждая ViewModel создала свой startItems(). Чьи данные правильные?

Две копии данных расходятся

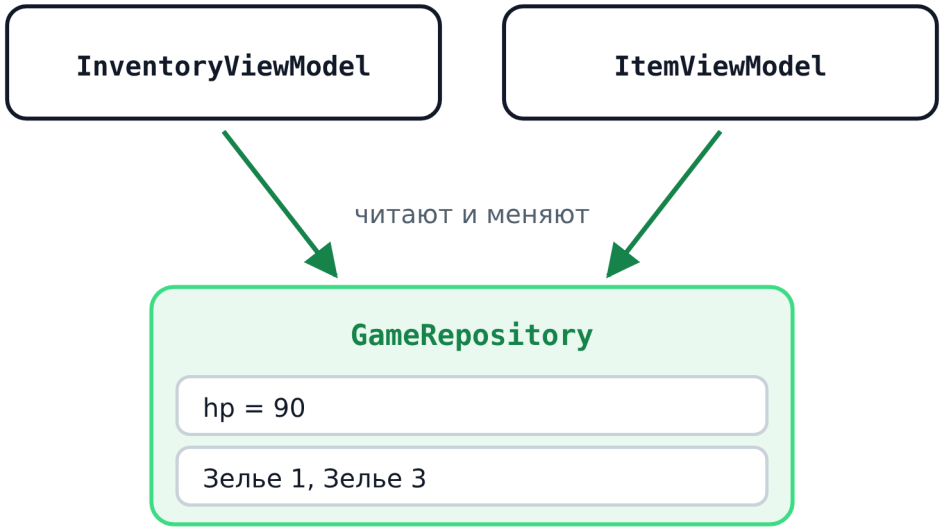
У каждой ViewModel своя копия



Использовали на карточке —
а в списке зелье осталось

Данные разошлись

Один репозиторий на приложение



Одни данные для всех экранов

Данные приложения нужны одному владельцу, а экраны читают их у него.

Репозиторий — единый источник правды

- Хранит данные приложения, а не одного экрана
- Один экземпляр на всё приложение
- ViewModel читают и меняют данные через него
- Экраны о репозитории не знают

ViewModel — состояние своего экрана. Репозиторий — данные приложения.

GameRepository

```
class GameRepository {
    var hp by mutableStateOf(MAX_HP)
        private set
    var items by mutableStateOf(startItems())
        private set

    fun item(id: Int): Item? = items.find { it.id == id }

    fun useItem(id: Int) {
        val item = item(id) ?: return
        hp = (hp + item.heal).coerceAtMost(MAX_HP)
        items = items - item
    }
}
```

Данные и правила работы с ними.

Compose-состояние: экраны увидят изменения.

damage() — так же.

Правило «вылечить и убрать предмет» переехало из ViewModel в репозиторий.

ItemViewModel читает репозиторий

```
class ItemViewModel(
    private val itemId: Int,
    private val repo: GameRepository,
) : ViewModel() {

    val uiState: ItemUiState
        get() = ItemUiState(
            item = repo.item(itemId),
            canUse = repo.hp < MAX_HP,
        )

    fun onUse() = repo.useItem(itemId)
}
```

uiState собирается из репозитория при каждом чтении.

Compose видит чтение hp и items и обновит экран.

ViewModel переводит данные приложения в состояние своего экрана.

Где создать единственный репозиторий

Где	Сколько живёт	Подходит?
remember в composable	Пока экран в композиции	Нет
ViewModel экрана	Пока ключ в стеке, у каждого свой	Нет
MainActivity	До поворота экрана	Нет
Application	Пока жив процесс	Да
object (синглтон)	Пока жив процесс	Работает, но доступен отовсюду

Репозиторий должен жить дольше любого экрана и любой Activity.

HeroApp: репозиторий на всё приложение

```
class HeroApp : Application() {
    val repository = GameRepository()
}

// AndroidManifest.xml
<application
    android:name=".HeroApp"
    ... >
```

Application создаётся раньше любой Activity.

Без строки в манифесте Android создаст обычный Application.

Свой класс Application регистрируем в манифесте через android:name.

Передаём зависимость параметром

```
class MainActivity : ComponentActivity() {
    override fun onCreate(state: Bundle?) {
        super.onCreate(state)
        val repo = (application as HeroApp).repository
        setContent {
            HeroTheme {
                HeroNavigation(repo)
            }
        }
    }
}
```

Достаём репозиторий в одном месте.

Дальше — только параметрами.

Это ручное внедрение зависимостей.

Кому нужен репозиторий, получает его снаружи, а не находит сам.

ItemRoute: id, репозиторий и выход

```
@Composable
fun ItemRoute(
    itemId: Int,
    repo: GameRepository,
    onBack: () -> Unit,
) {
    val viewModel = viewModel {
        ItemViewModel(itemId, repo)
    }
    ItemScreen(
        state = viewModel.uiState,
        onUse = { viewModel.onUse(); onBack() },
        onBack = onBack,
    )
}
```

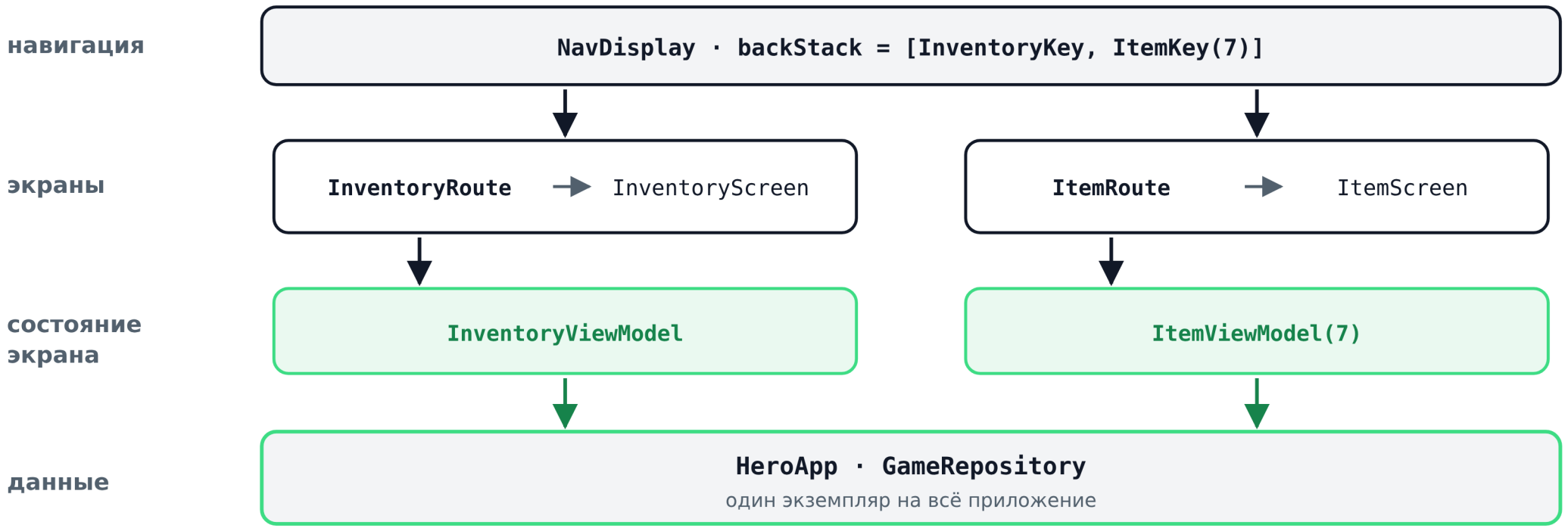
itemId — из ключа.

repo — из навигации.

После использования предмета — назад. ViewModel о навигации не знает.

Route получает зависимости и переходы параметрами. ItemScreen — только state и колбэки.

Схема приложения



Навигация → экраны → ViewModel экранов → один репозиторий.

Ключ есть, а предмета нет

```
@Composable
fun ItemScreen(
    state: ItemUiState,
    onUse: () -> Unit,
    onBack: () -> Unit,
) {
    val item = state.item
    if (item == null) {
        NotFound(onBack = onBack)
    } else {
        ItemDetails(item, state.canUse, onUse, onBack)
    }
}
```

Стек сохраняется, данные в памяти — нет.

После завершения процесса ключ может указывать на то, чего нет.

Экран по ключу должен уметь показать «не найдено».

Кто за что отвечает

Часть	Отвечает за
HeroApp	Создаёт единственный GameRepository
GameRepository	Данные приложения и правила их изменения
HeroNavigation	Стек экранов и переходы
…Route	ViewModel экрана, связь с навигацией
…ViewModel	Состояние своего экрана
…Screen и компоненты	Показывают state, сообщают о событиях

Каждое знание — в одном месте.

Частые ошибки

Симптом	Что проверить
После поворота снова первый экран	Стек в remember, а не в rememberNavBackStack
Карточка показывает прошлый предмет	Нет rememberViewModelStoreNavEntryDecorator()
Изменения не видны на другом экране	У ViewModel своя копия данных, а не репозиторий
Данные сбрасываются при повороте	Репозиторий создан в Activity или composable
Ошибка про неизвестный ключ	Нет entry<...> для этого ключа
ClassCastException на HeroApp	Не указан android:name в манифесте
Экран сам меняет backStack	Передан стек вместо колбэков

Сначала спрашиваем: где живёт это состояние и кто его владелец?

Проверяем понимание

- Что такое стек экранов в Navigation 3 и кто его меняет?
- Почему в ItemKey лежит id, а не сам Item?
- Что изменится без rememberViewModelStoreNavEntryDecorator?
- Почему репозиторий создаётся в Application, а не в Activity?
- Почему ItemScreen получает onBack, а не backStack?

Ответ должен объяснять поведение программы, а не только название API.

Теперь смотрим целое приложение

- Список предметов и карточка предмета
- Поворот на карточке: экран на месте
- Использовали зелье — в списке его нет, hp выpос
- Без декоратора: две карточки, одна ViewModel



Сначала показываем готовый проект, затем ломаем его по одному месту.

Практика: третий экран

- Добавить экран персонажа: имя, здоровье, урон
- Открывать его из списка предметов
- Урон на экране персонажа виден в списке
- Экран переживает поворот, «назад» возвращает в список

Проверить: поворот на каждом экране и изменения после возврата.

Что дальше

- Корутины и viewModelScope
- StateFlow в репозитории вместо Compose-состояния
- Хранение данных на устройстве: Room и DataStore
- Внедрение зависимостей библиотекой

Схема «экран → ViewModel → репозиторий» останется. Изменится то, как данные текут.

Материалы для повторения

- Navigation 3: overview, basics, save state
- Navigation 3 recipes (GitHub)
- Create ViewModels with dependencies
- Data layer, Manual dependency injection

Ссылки на соответствующие темы есть в заметках к слайдам.